Llamadas a Procedimientos Remotos (RPC)

Alvaro Ospina S

2026

Conceptos que deben estar claros

- ¿Qué es un Middleware?
- ¿Qué tipos o clases de Middlewares hay?
- Diseño de aplicaciones distribuidas dependiendo del tipo de middleware.
- Serialización
- Ejemplo:
  - Con el desarrollo de aplicaciones basadas en middleware de comunicaciones como sockets se requiere tener en cuenta?
    - Debe tener en cuenta las consideraciones del diseño de sistemas distribuidos y protocolos, además, en un MW de invocación remota se debe tener en cuenta:
      - Envío/Recepción de tipos de datos
      - Diferentes tipos de codificación de tipos de datos
      - Identificación de componentes del sistema distribuido (objetos, procedimientos, componentes)
      - Mecanismos de activación de objetos en el servidor.
      - Seguridad en tipos de datos (type Safety)
      - Implementación de mecanismos de sincronización en TCP y UDP

Serialización en Sistemas Distribuidos

En sistemas distribuidos, los componentes suelen estar en diferentes máquinas o procesos, y necesitan intercambiar datos. La serialización convierte los objetos o estructuras de datos en un formato que puede ser enviado a través de la red (por ejemplo, JSON, XML, Protocol Buffers, etc.). El receptor deserializa los datos para reconstruir el objeto o estructura original.

Importancia de la Serialización

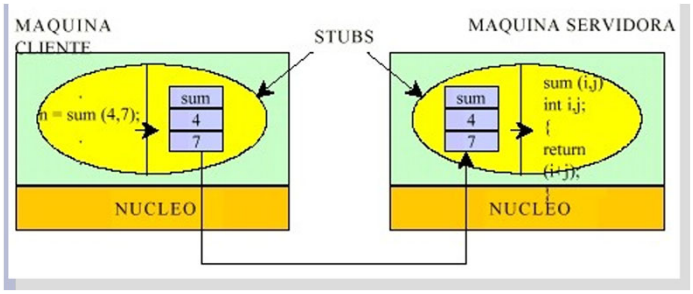
- **Interoperabilidad**: Permite que sistemas escritos en diferentes lenguajes se comuniquen.
- **Eficiencia**: Algunos formatos son más compactos y rápidos que otros.
- **Compatibilidad**: Facilita la evolución de las APIs y sistemas sin romper la compatibilidad.

.

Llamadas a Procedimientos Remotos (RPC)

En que consiste el modelo de llamadas remotas

- Permitir que un cliente, pueda realizar llamadas a procedimientos, funciones o métodos en otra parte de la red.
- Para que?
- Que retos enfrente un middleware que soporte este paradigma?

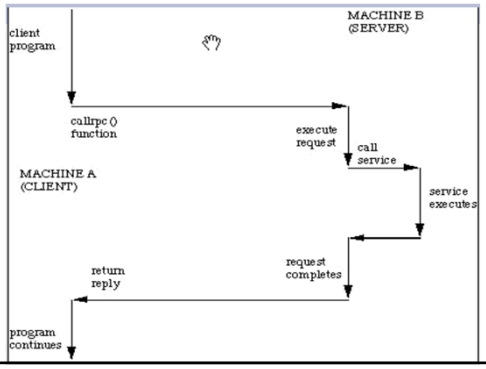


Llamadas a procedimientos remotos RPC

- Nace de la época de la programación procedimental.
- Trata de simular la misma semántica de una invocación local.
- La idea con RPC es hacer llamados a procedimientos localizados en otras máquinas.

Llamadas a procedimientos remotos

- Funcionamiento:
  - Un proceso en una maquina A llama a un proceso en la maquina B, el proceso en A se bloquea mientras que se ejecuta el proceso en B.
  - La información que se intercambia son parámetros y se espera respuesta.
- Problemas:
  - Espacio de direccionamiento diferente
  - Diferentes tipos de datos
  - Fallas



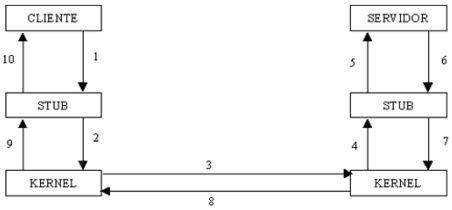
Ejecución de una llamada local

- por ejemplo la llamada invocada desde el *main()*:
  res = sumar(a, b);
- El llamador coloca los parámetros en el stack en orden, último primero.
- se ejecuta la función “sumar”
- se coloca el valor de retorno en un registro, remueve la dir. de retorno y transfiere de nuevo el control al llamador.
- el llamador remueve los parametros del stack y sigue ejecutando otras instrucciones.
- Paso de parámetros:
  - Por VALOR
  - Por REFERENCIA

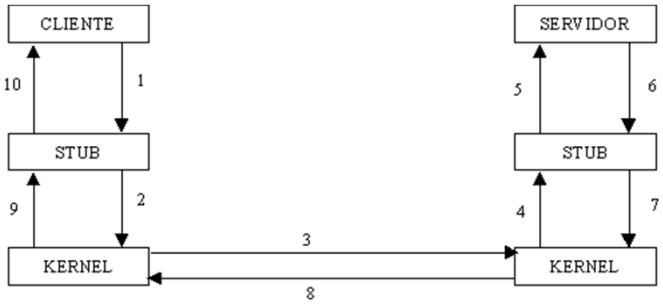
Funcionamiento de RPC

- La idea con RPC es hacer ver a una llamada remota como si fuera local, por esto la invocación debe ser transparente para el que la utiliza.
- La transparencia en RPC se logra agregando un Stub o proxy tanto al cliente como al servidor y utilizando un IDL
- Los pasos que ejecuta un Cliente para invocar un procedimiento remoto en un Servidor son los siguientes:

Funcionamiento de RPC



1. El Cliente llama un procedimiento local llamado el Client Stub (proxy).
2. El Client Stub empaqueta (marshalling) los parámetros, construye un mensaje y lo envia al kernel.
3. El kernel local lo envia al kernel remote donde se encuentra el Server Stub
4. El kernel remoto envía el mensaje al Server Stub.
5. El Server Stub desempaqueta (unmarshalling) los parámetros, identifica el procedimiento y lo ejecuta.
6. El Server Stub recibe el resultado del servidor
7. El Server Stub empaqueta (marshalling) la respuesta, construye un mensaje y lo envia al Kernel.
8. El Kernel remoto lo envia de nuevo al kernel local(cliente).
9. El Kernel local envía el mensaje al Client Stub.
10. Client Stub desempaqueta (marshalling) el resultado y lo retorna de la misma forma que un procedimiento local.

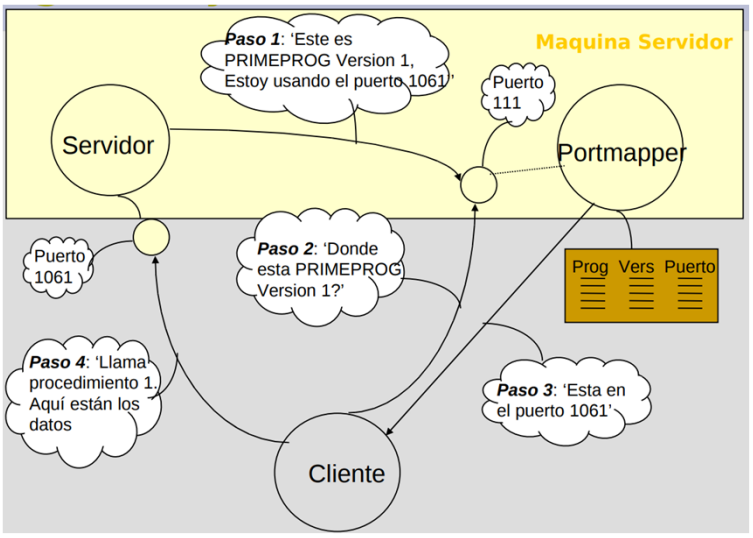


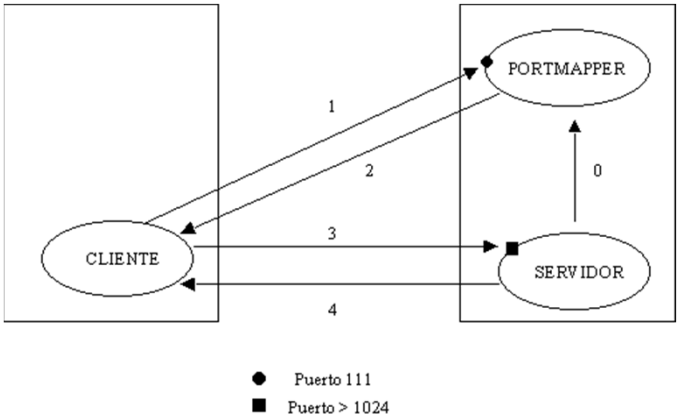
Paso de parámetros en RPC

- Debido a problemas de alfabetos (ASCII/ABCDIC), representación de enteros (complemento a uno o a dos), punto flotante, ordenamiento de bytes (Little Endian y Big Endian), se hace necesario establecer una forma canonica de representar los distintos tipos de datos.
- Se ejecuta un proceso conocido como Marshalling/UnMarshalling, el cual consiste el transformar los tipos de datos locales de una maquina en un formato estándar para ser transmitidos por la red.
- Los tipos básicos como escalares entre otros se pasan por valor, cuando son arreglos tanto en los clientes como servidores se realiza una copia local, se manipulan y se envian por la red.
- En RPC no hay paso de punteros.
- Además de los parámetros normales de una función, se requiere transferir otra información como: Nombre del procedimiento, versión, etc.

Localización de Servidores

- Como un cliente localiza un servidor?
  1) podría ser que el cliente tuviera la dir. del servidor. Esto es muy ineficiente y estático.
  2) realizar un proceso dinámico de asociación o binding con el servidor.
- Partimos de una especificación formal del servidor:
  - nombre del servidor
  - número de versión
  - lista de procedimientos
  - para cada procedimiento se especifica los parámetros
- La especificación formal es muy util para los generadores de código. Cuando un servidor comienza a ejecutar envía un mensaje a un programa (local o remoto) llamado el "binder" o servidor de registro.
- El cliente conoce la dirección del binder a quien interroga por la existencia o no de los servidores que desea contactar.
- Este binder se conoce como un Localizador de Recursos en la red, un broker o un "corredor" de procedimientos.


Maquina Servidor
Paso 1: 'Este es PRIMEPROG Version 1, Estoy usando el puerto 1061'
Puerto 111
Servidor
Portmapper
Puerto 1061
Paso 2: 'Donde esta PRIMEPROG Version 1?'
Prog Vers Puerto
Paso 4: 'Llama procedimiento 1. Aquí están los datos
Paso 3: 'Esta en el puerto 1061'
Cliente


PORTMAPPER
1
2
0
CLIENTE
3
SERVIDOR
4
Puerto 111
Puerto > 1024

¿Por qué necesitamos un IDL en RPC?

- Se debe separar la especificación de la implementación de los RPC
- Se debe hacer en un lenguaje neutral.
- Para el caso de SUN-RPC existe un lenguaje conocido como RPCL
- En este se especifican los procedimientos
- Se precompilan para generar stubs, proxies, clientes y servidores.

¿Por qué necesitamos un IDL en RPC?

- Separa la especificación de la implementación.
- Define interfaces en un lenguaje neutral.
- Ejemplo: SUN-RPC usa RPCL como IDL.
- IDL describe procedimientos y parámetros.
- Genera automáticamente:
  - Stubs (cliente - empaquetar)
  - Proxies (oculta complejidad de comunicación)
  - Skeletons (servidor - desempaquetar)

Aclaraciones …

- rpcbind es el reemplazo moderno de portmap y es necesario para sistemas con **TI-RPC** y soporte para IPv6 y seguridad avanzada.
- Si tu código usa RPC tradicional, debes migrarlo a TI-RPC y asegurarte de que rpcbind esté en ejecución.

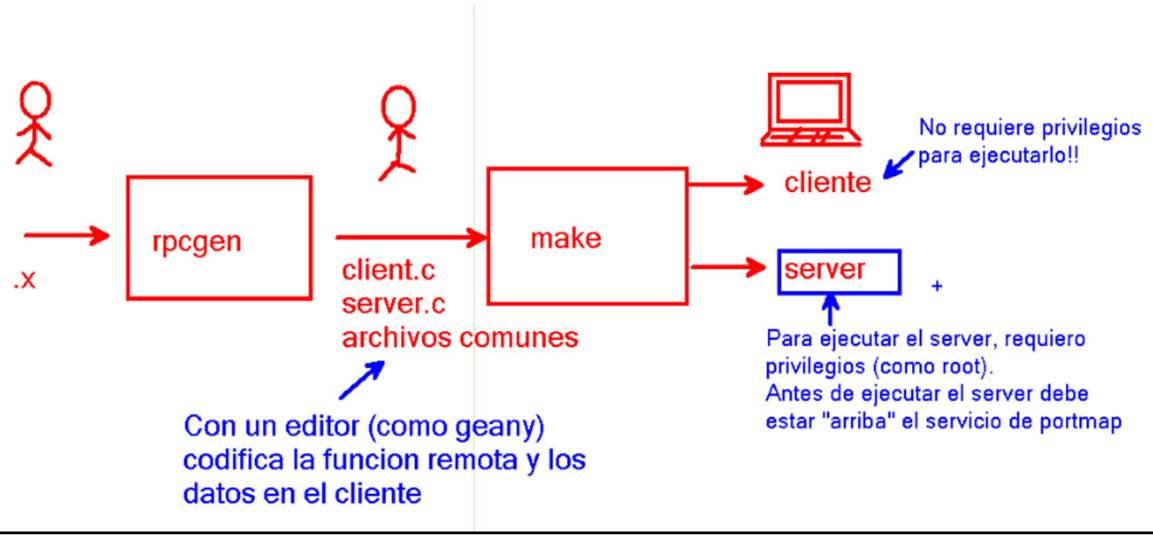
Aclaraciones …

Característica	portmap (RPC tradicional)	rpcbind (TI-RPC)
Soporte de transporte	Solo IPv4	IPv4 e IPv6
Protocolo	PORTMAPPER (ID: 100000)	RPCBIND (ID: 100000)
Autenticación	No soporta	Soporta GSS-API (Kerberos)
Disponibilidad	Obsoleto	Reemplazo moderno
Seguridad	Baja (sin autenticación)	Mejor seguridad y control de acceso
Uso en Ubuntu 22.04	Eliminado	Obligatorio para TI-RPC

Aclaraciones …

Característica	RPC tradicional	TI-RPC
Dependencia de transporte	UDP/TCP (BSD Sockets)	Soporta múltiples protocolos (IPv4, IPv6, SCTP, etc.)
Librería	librpcsvc	libtirpc
Manejador de puertos	portmap	rpcbind (versión extendida del PORTMAPPER)
Soporte para IPv6	No	Sí
Seguridad	Limitada	Soporta autenticación avanzada (Kerberos)
Compatibilidad	Funciona en sistemas antiguos	Optimizado para sistemas modernos

Un ejemplo en RPCL



Un ejemplo en RPCL

sumador.x

```
1  struct sumandos {
2    int sumando1;
3    int sumando2;
4  };
5
6  program PROGRAMA_SUMA {
7    version VERSION_SUMA {
8      int suma(sumandos) = 1;
9      int resta(sumandos) = 2;
10   } = 1;
11 } = 0x20000001;
12
```

Server (sin main)

```
1  include "sumador.h"
2
3  int *
4  suma_1_svc(sumandos *argp, struct svc_req *rqstp)
5  {
6    static int  result;
7
8    /** insert server code here */
9    result = argp->sumando1 + argp->sumando2;
10
11   return &result;
12 }
13
14 int *
15 resta_1_svc(suamandos *argp, struct svc_req *rqstp)
16 {
17   static int  result;
18
19   /** insert server code here */
20   result = argp->suamando1 - argp->sumando2;
21
22   return &result;
23 }
```

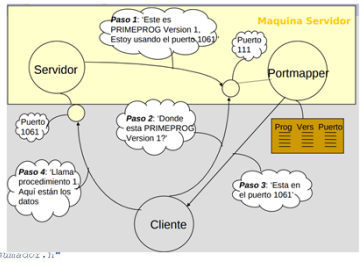
Client

```
#include "sumador.h"

void
programa_suma_1(char *host)
{
    CLIENT *clnt;
    int  *result_1;
    sumandos  suma_1_arg;
    int  *result_2;
    sumandos  resta_1_arg;

    fprintf(stderr, "Cambiando los parametros \n");
   /** Parametros de la suma **/
   suma_1_arg.sumando1 = 5;
   suma_1_arg.sumando2 = 8;

    /** Parametros de la resta **/
    resta_1_arg.sumando1 = 10;
    resta_1_arg.sumando2 = 3;

#ifndef DEBUG
    clnt = clnt_create (host, PROGRAMA_SUMA, VERSION_SUMA, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif  /* DEBUG */
```

Un ejemplo en

sin titulo

```
1  apt install
2
3  build-essential  (gcc)
4  rpcbind          (servicio old-portmap)
5  libc6-dev           (rpcgen)
6  net-tools           (netstat)
7  inetutils-ping      (ping)
8  nano                (editor de texto)
9  libtirpc-dev        (rpcbind)
```



sumador.x

```
1  struct sumandos {
2    int sumando1;
3    int sumando2;
4  };
5
6  program PROGRAMA_SUMA {
7    version VERSION_SUMA {
8      int suma(sumandos) = 1;
9      int resta(sumandos) = 2;
10   } = 1;
11 } = 0x20000001;
12
```

sumador_Server.c (sin main)

```
1  include "sumador.h"
2
3  int *
4  suma_1_svc(sumandos *argp, struct svc_req *rqstp)
5  {
6    static int  result;
7
8    /** insert server code here */
9    result = argp->sumando1 + argp->sumando2;
10
11   return &result;
12 }
13
14 int *
15 resta_1_svc(suamandos *argp, struct svc_req *rqstp)
16 {
17   static int  result;
18
19   /** insert server code here */
20   result = argp->suamando1 - argp->sumando2;
```

```
#include "sumador.h"

void
programa_suma_1(char *host)
{
    CLIENT *clnt;
    int  *result_1;
    sumandos  suma_1_arg;
    int  *result_2;
    sumandos  resta_1_arg;

    fprintf(stderr, "Cambiando los parametros \n");
   /** Parametros de la suma **/
   suma_1_arg.sumando1 = 5;
   suma_1_arg.sumando2 = 8;

    /** Parametros de la resta **/
    resta_1_arg.sumando1 = 10;
    resta_1_arg.sumando2 = 3;

#ifndef DEBUG
    clnt = clnt_create (host, PROGRAMA_SUMA, VERSION_SUMA, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif  /* DEBUG */
```

```
3  * docker run -itd -v ./mpi:/home/ubuntu/mpi ubuntu bash
4  * docker exec -it 29 bash
```

Un ejemplo en RPCL

archivo.x

```
1 struct sumandos {
2   int sumando1;
3   int sumando2;
4 };
5
6 program PROGRAMA_SUMA {
7   version VERSION_SUMA {
8     int suma(sumandos) = 1;
9     int resta(sumandos) = 2;
10  } = 1;
11 } = 0x20000001;
12
```

Server (sin main)

```
1 include "sumador.h"
2
3 int *
4 suma_1_svc(sumandos *argp, struct svc_req *rqstp)
5 {
6   static int  result;
7
8   /** insert server code here */
9   result = argp->sumando1 + argp->sumando2;
10
11  return &result;
12 }
13
14 int *
15 resta_1_svc(suamandos *argp, struct svc_req *rqstp)
```

```
root@localhost:~/sis
Archivo  Editar  Ver  Terminal  Solapas  Ayuda
[root@localhost sis]# ls
sumador.x
[root@localhost sis]# rpcgen -a sumador.x
[root@localhost sis]# ls
Makefile.sumador  sumador.h          sumador.x
sumador_client.c  sumador_server.c   sumador_xdr.c
sumador_clnt.c    sumador_svc.c
[root@localhost sis]#
```

```
result;

erver code here */
p->suamandol - argp->sumando2;

lt;
23 }
```

Client

```
#include "sumador.h"

void
programa_suma_1(char *host)
{
    CLIENT *clnt;
    int  *result_1;
    sumandos  suma_1_arg;
    int  *result_2;
    sumandos  resta_1_arg;

    fprintf(stderr, "Cambiando los parametros \n");
   /** Parametros de la suma **/
   suma_1_arg.sumando1 = 5;
   suma_1_arg.sumando2 = 8;

    /** Parametros de la resta **/
    resta_1_arg.sumando1 = 10;
    resta_1_arg.sumando2 = 3;

#ifndef DEBUG
    clnt = clnt_create (host, PROGRAMA_SUMA, VERSION_SUMA, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif  /* DEBUG */
```

Un ejemplo en RPCL

archivo.x

```
1 struct sumandos {
2   int sumando1;
3   int sumando2;
4 };
5
6 program PROGRAMA_SUMA {
7   version VERSION_SUMA {
8     int suma(sumandos) = 1;
9     int resta(sumandos) = 2;
10  } = 1;
11 } = 0x20000001;
12
```

Server (sin main)

```
1 include "sumador.h"
2
3 int *
4 suma_1_svc(sumandos *argp, struct svc_req *rqstp)
5 {
6   static int  result;
7
8   /** insert server code here */
9   result = argp->sumando1 + argp->sumando2;
10
11  return &result;
12 }
13
14 int *
15 resta_1_svc(suamandos *argp, struct svc_req *rqstp)
16 {
17  static int  result;
18
19  /** insert server code here */
20  result = argp->suamandol - argp->sumando2;
21
22  return &result;
23 }
```

Client

```
#include "sumador.h"

void
programa_suma_1(char *host)
{
    CLIENT *clnt;
    int  *result_1;
    sumandos  suma_1_arg;
    int  *result_2;
    sumandos  resta_1_arg;

    fprintf(stderr, "Cambiando los parametros \n");
   /** Parametros de la suma **/
   suma_1_arg.sumando1 = 5;
   suma_1_arg.sumando2 = 8;

    /** Parametros de la resta **/
    resta_1_arg.sumando1 = 10;
    resta_1_arg.sumando2 = 3;

#ifndef DEBUG
    clnt = clnt_create (host, PROGRAMA_SUMA, VERSION_SUMA, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);
        exit (1);
    }
#endif  /* DEBUG */
```

Un ejemplo en RPCL

```
root@localhost:~/sis
Archivo  Editar  Ver  Terminal  Solapas  Ayuda
[root@localhost sis]# make -f Makefile.sumador
cc -g     -c -o sumador_clnt.o sumador_clnt.c
cc -g     -c -o sumador_client.o sumador_client.c
cc -g     -c -o sumador_xdr.o sumador_xdr.c
cc -g      -o sumador_client  sumador_clnt.o sumador_clie
nt.o sumador_xdr.o -lnsl
cc -g     -c -o sumador_svc.o sumador_svc.c
cc -g     -c -o sumador_server.o sumador_server.c
cc -g      -o sumador_server  sumador_svc.o sumador_serve
r.o sumador_xdr.o -lnsl
[root@localhost sis]# clear

[root@localhost sis]# make -f Makefile
make: No se hace nada para `all'.
[root@localhost sis]# ls -al
```

```
    int resta(sumandos) = 2;
  } = 1;
} = 0x20000001;
```

Client

```
#include "sumador.h"

void
programa_suma_1(char *host)
{
    CLIENT *clnt;
```

```
c_req *rqstp)
```

```
root@localhost:~/sis
Archivo  Editar  Ver  Terminal  Solapas  Ayuda
[root@localhost sis]# ./sumador_server
```

```
root@localhost:~/sis
Archivo  Editar  Ver  Terminal  Solapas  Ayuda
[root@localhost sis]# ./sumador_client localhost
Cambiando los parametros
Resultado de la suma: 13
Resultado de la resta: 7
[root@localhost sis]#
```

Un ejemplo en RPCL

```
root@orex2: ~/distribuidos/rpc/ej1 - Terminator
root@orex2:~/distribuidos/rpc/ej1# pmap_dump
    100000    2   tcp    111  portmapper
    100000    2   udp    111  portmapper
 536870913    1   udp    771
 536870913    1   tcp    772
root@orex2:~/distribuidos/rpc/ej1# netstat -nat
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:772             0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 192.168.0.102:22        192.168.0.10:3230       ESTABLISHED
tcp        0      0 127.0.0.1:58022         127.0.0.1:111           TIME_WAIT
tcp        0      0 127.0.0.1:39472         127.0.0.1:111           TIME_WAIT
tcp        0      0 127.0.0.1:52201         127.0.0.1:111           TIME_WAIT
tcp        0      0 127.0.0.1:56519         127.0.0.1:111           TIME_WAIT
tcp        0      0 192.168.0.102:22        192.168.0.10:3146       ESTABLISHED
tcp        0      0 127.0.0.1:42831         127.0.0.1:111           TIME_WAIT
tcp6       0      0 :::22                   :::*                    LISTEN
root@orex2:~/distribuidos/rpc/ej1#
```

```
struc
  int
  int
};

progr
  ver
    i
    i
  } =
} = 0
```

```
} 
```

RPCs retos/oportunidades

- Que limitaciones ven en RPC

RPCs precursores en la era Web

- Propuestas como XML-RPC y Web Services son versiones modernas de RPC.
- Más adelante se detallarán los Web Services por su importancia hoy en día

¿Qué es gRPC?

- Framework de comunicación RPC (Remote Procedure Call)
- Desarrollado por Google
- Basado en HTTP/2, usa Protocol Buffers (protobuf) como IDL

¿Por qué usar gRPC?

- Comunicación eficiente y rápida
- Soporte para múltiples lenguajes
- Streaming de datos
- Generación automática de código por IDL

gRPC vs REST

gRPC:
- Usa HTTP/2
- Basado en Protobuf
- Mayor velocidad
- Soporta streaming

REST:
- Usa HTTP/1.1
- Basado en JSON
- Menor velocidad
- Streaming limitado

Conclusión gRPC

- gRPC es eficiente y rápido
- Útil para microservicios y comunicación entre sistemas
- Soporta múltiples lenguajes y streaming